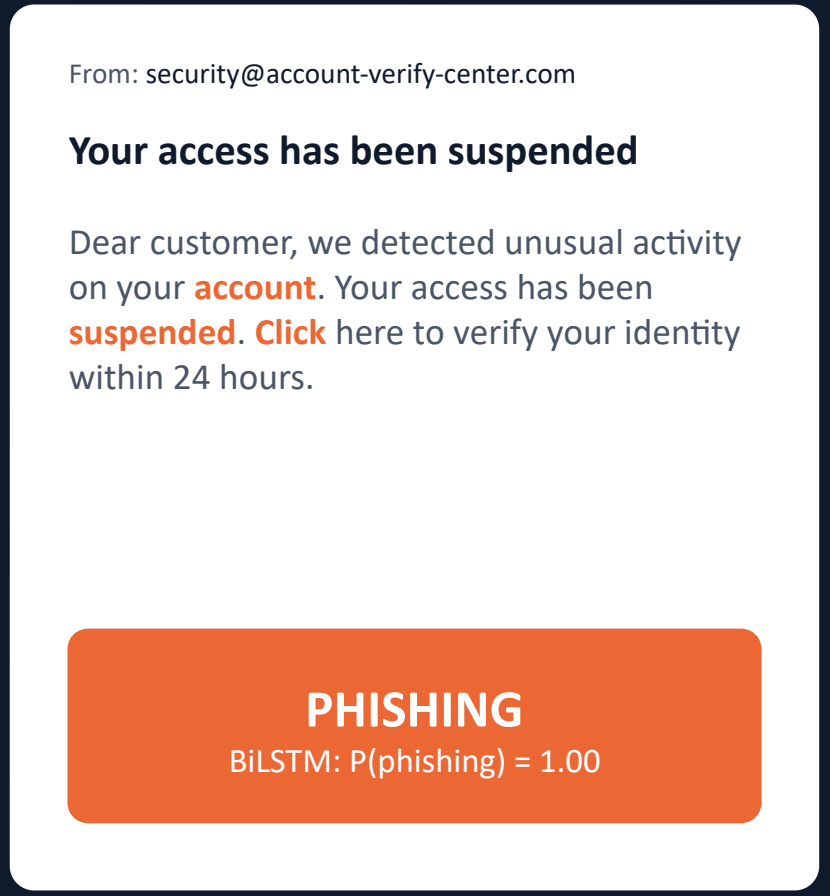
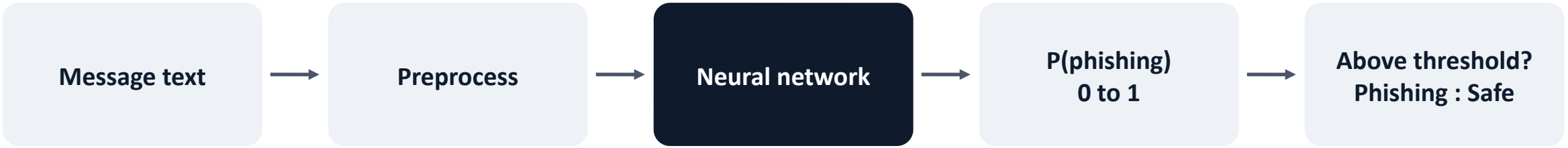
Phishing Message Detection with Deep Learning

Training text classifiers on labelled messages and studying their false positives and false negatives

Artificial Intelligence · Course Project
[Team member names]



A wrong answer is costly either way



FP

False positive

A safe message is flagged as phishing.
Legitimate mail is lost and users stop trusting the filter.

Our best model on the test set: **18 of 1,646 safe test messages (1.1%)**

FN

False negative

A phishing message gets through.
The user may lose a password, money or their account.

Our best model on the test set: **16 of 981 phishing test messages (1.6%)**

17,511 unique labelled emails

18,650 → 17,511

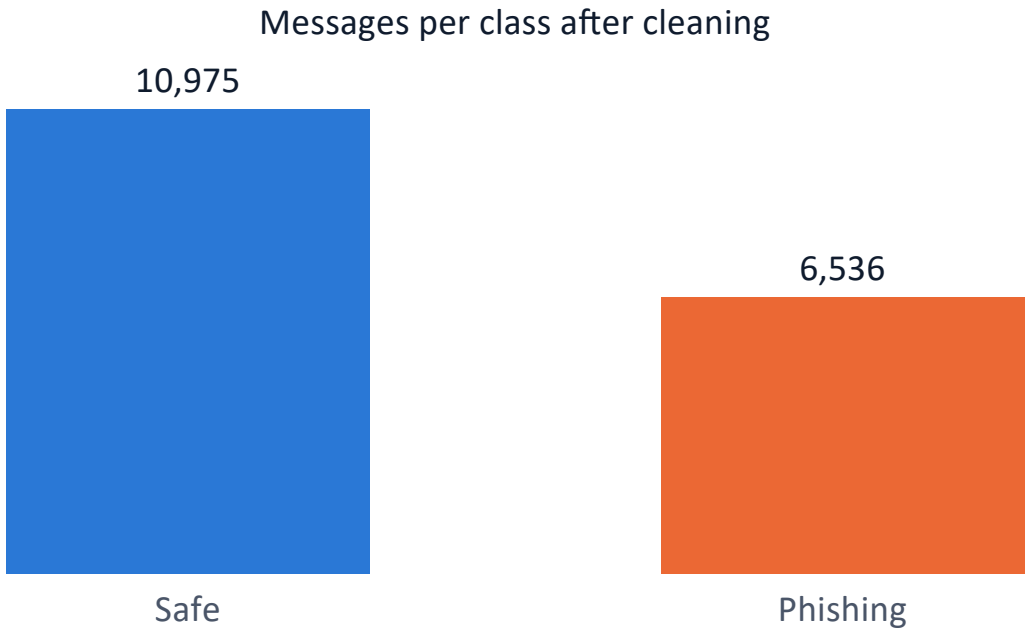
emails after removing empty rows and duplicates

577

exact duplicates removed so no message is in both train and test (no leakage)

37%

phishing (6,536) versus 63% safe (10,975)



Train · 70%
12,257
learn weights + vocabulary

Validation · 15%
2,627
early stopping, threshold

Test · 15%
2,627
final evaluation, used once

From raw text to numbers

1

Normalise

Lower-case; URLs → urltoken, emails → emailtoken, money → moneytoken, numbers → numtoken

2

Tokenise

Split into words; keep ! ? $ % as tokens (urgency and money are phishing signals)

3

Build vocabulary

30,000 most frequent words from the training set only; unknown words → <unk>

4

Fix the length

Keep the first 300 tokens: the whole message for 75% of emails

Raw message

```
URGENT! Verify your account at http://secure-
login.com within 24 hours or pay $500
```

↓

Tokens

```
urgent ! verify your account at urltoken
within numtoken hours or pay moneytoken
```

↓

Token IDs (the network's input)

```
[3141, 18, 2855, 25, 306, 30, 53, 179, 2, 559,
23, 457, 33]
```

Three models, from classical to deep

A TF-IDF + LogReg

Baseline: bag of words

- TF-IDF weights for 60k words and word pairs
- Logistic regression: one weight per feature
- Fast, interpretable reference point

Trains in seconds

B TextCNN

Deep learning: learned phrase detectors

- 128-d word embeddings
- 384 convolution filters spanning 3, 4 and 5 words
- Max-pooling keeps the strongest match

4.04M parameters · 20 s/epoch

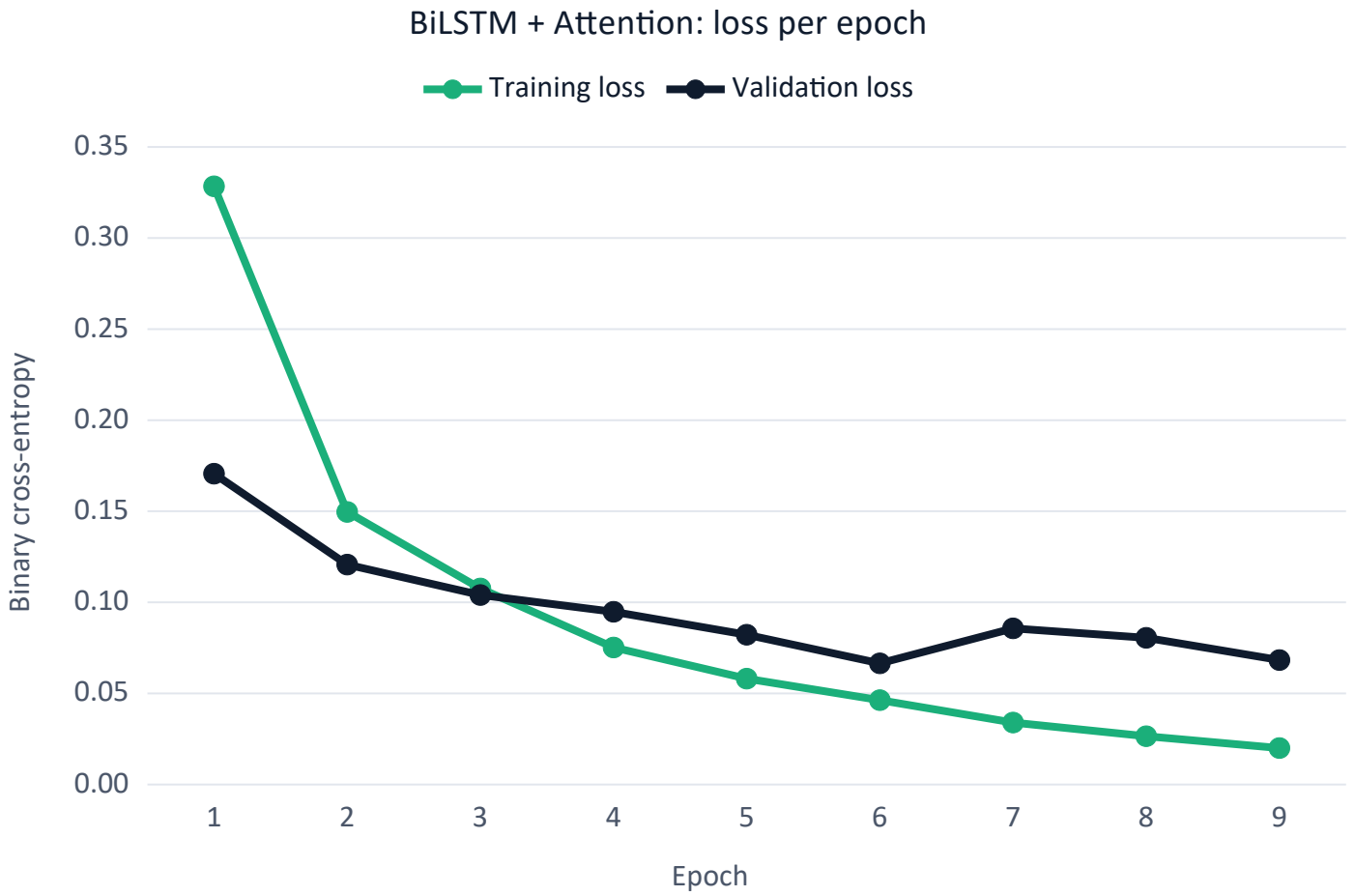
C BiLSTM + Attention

Deep learning: reads the sequence both ways

- 128-d word embeddings
- Bidirectional LSTM, 128 units each way
- Attention weights every word, so decisions can be explained

4.10M parameters · 4 min/epoch

Early stopping caught overfitting



BiLSTM

Best validation loss at epoch 6 (0.067). Training loss kept falling while validation loss rose, so training stopped at epoch 9 and the epoch-6 weights were kept.

Setup (both models)

- AdamW, learning rate 0.001, batch 64
- Binary cross-entropy loss
- Dropout + gradient clipping
- Stop after 3 epochs without improvement

All models reach about 98% accuracy

Model (threshold 0.5)	Accuracy	Precision	Recall	F1	False pos.	False neg.
TF-IDF + LogReg (baseline)	98.59%	98.56%	97.66%	0.981	14	23
TextCNN	98.40%	98.65%	97.04%	0.978	13	29
BiLSTM + Attention	97.79%	96.01%	98.17%	0.971	40	18
Ensemble (average of all 3)	**98.71%**	**98.17%**	**98.37%**	**0.983**	**18**	**16**

98.7%

Ensemble accuracy, the best result (F1 0.983)

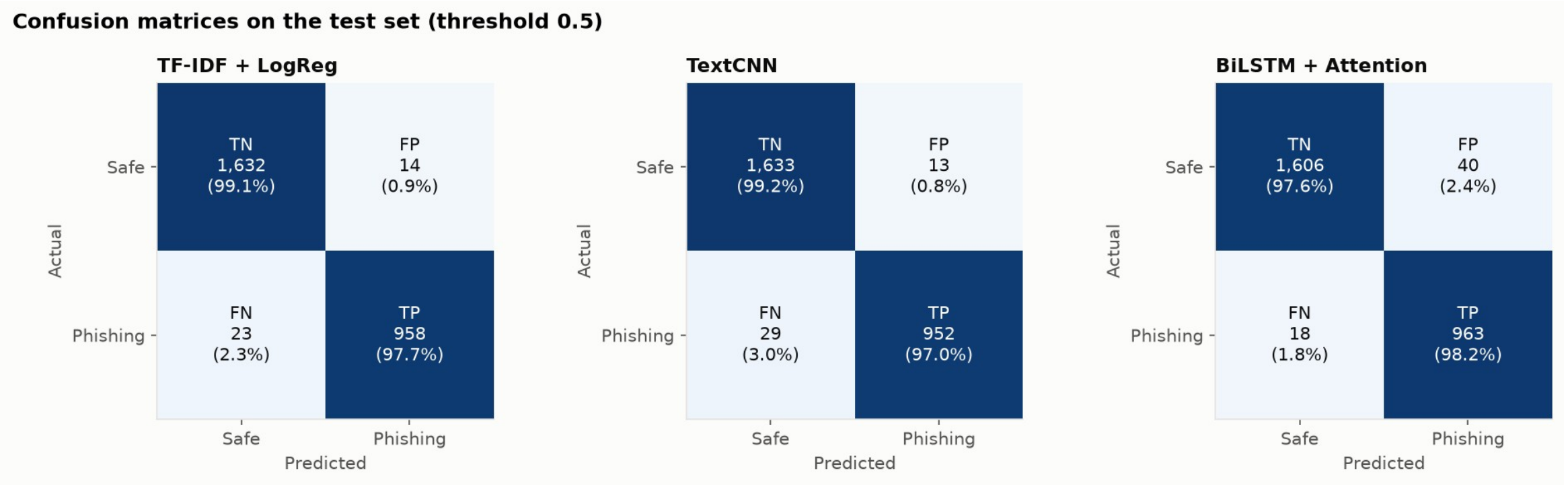
13

False alarms for TextCNN, the fewest of any model

0.981

Baseline F1, above TextCNN (0.978): deep learning alone did not win

Where each model goes wrong

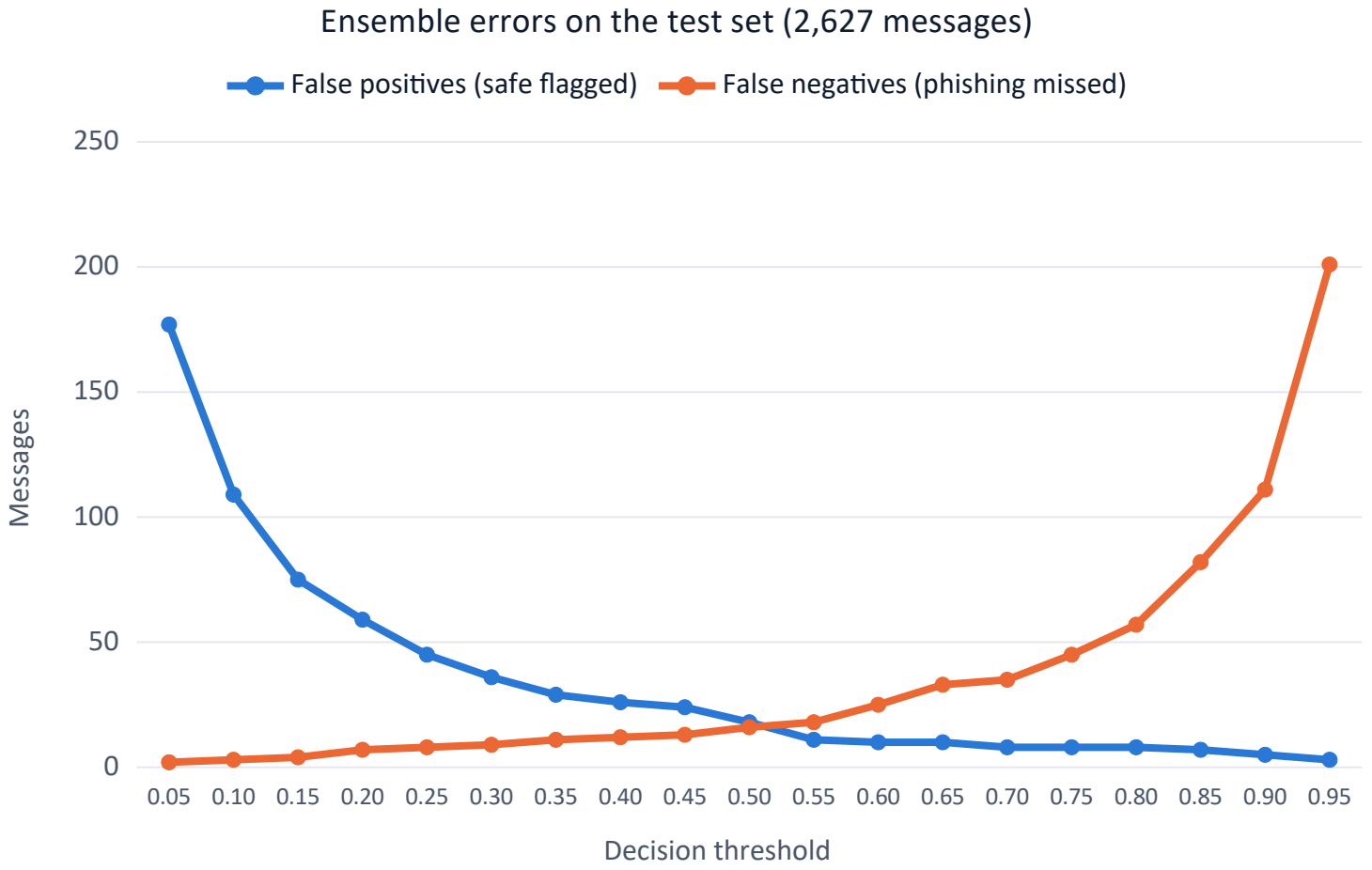


A **Baseline**
Balanced: 14 false alarms, 23 misses

B **TextCNN**
Fewest false alarms (13), most misses (29)

C **BiLSTM**
Fewest misses (18), but 40 false alarms

The threshold trades FPs for FNs



Default threshold 0.50

18 FP · 16 FN

Balanced

Cost-tuned threshold 0.21

54 FP · 7 FN

Recall 99.3%: misses cut by 56%

0.21 minimises 3 × FN + 1 × FP on the validation set: a missed phish is assumed 3× worse than a false alarm. Where to set it is a policy choice, not a modelling one.

False positives: real mail, phishy words

P(phishing) = 1.00

Order confirmation

“thank you for shopping at radioshack.com. your order number is … pending credit card approval”

Phishing-like cues: credit card, order number, link

P(phishing) = 1.00

Domain registration

“thank you for using register.com dear louise … a host of helpful tools and services”

Phishing-like cues: “thank you for using”, offers, web addresses

P(phishing) = 1.00

Newsletter sign-up

“welcome to energy news live … you are receiving this email as a result of your recent registration”

Phishing-like cues: “you are receiving this email”, links

All three models flag these with full confidence. Transactional and marketing mail uses phishing's vocabulary (orders, credit cards, links, money). Text alone cannot separate them; real filters also check the sender (SPF / DKIM) and where links lead.

Why phishing slipped through

1 Label noise

P = 0.01

“junk mail: books for linguists … has a new catalogue”

The 'phishing' class also holds plain spam and newsletters

2 Too little text

P = 0.25

“Could you please send the address of your agent in Saudi Arabia. Thanks”

Under 30 tokens: the highest error rate (3 to 5%)

3 Casual social engineering

P = 0.17

“hey, what's up. here's the link that you requested …”

No alarm words: it imitates a friend

4 New attack style

P = 0.25

“URGENT: Your mailbox has exceeded its storage limit. Login at …”

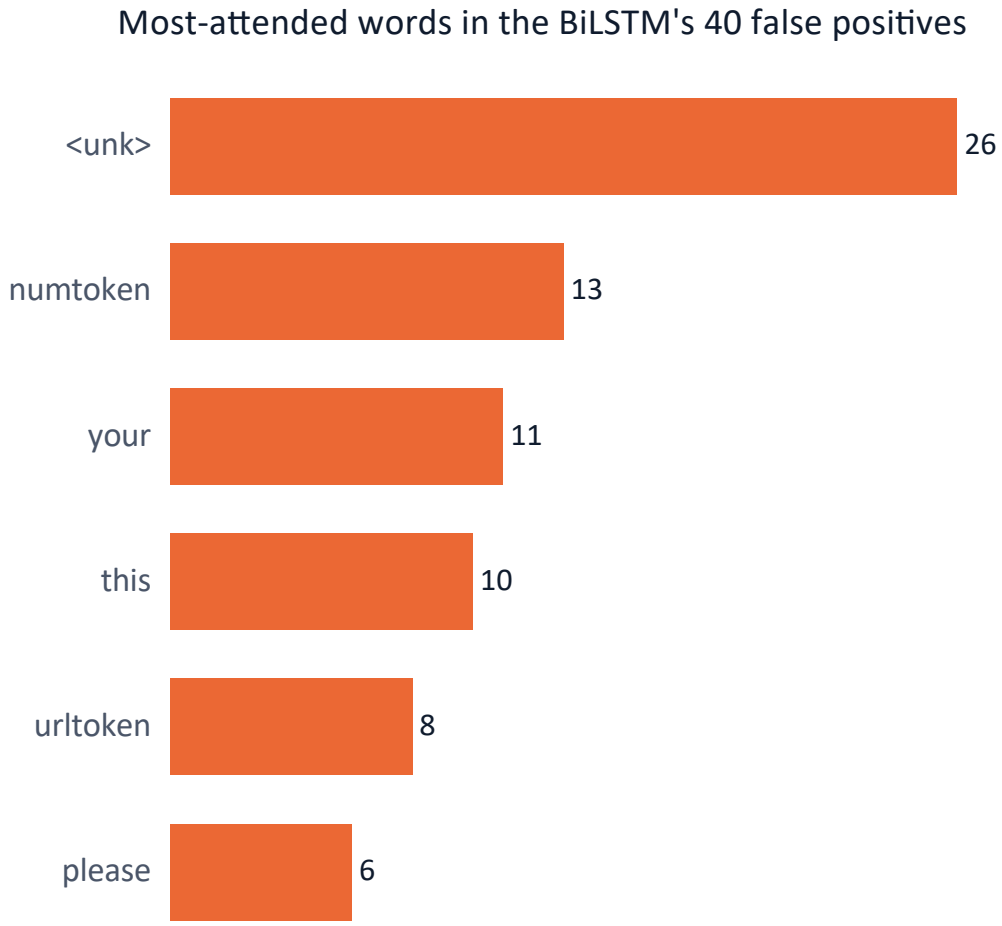
Training data is mostly older spam; this lure is rare in it

What the BiLSTM pays attention to

"…unusual activity on your **account**. Your access has been **suspended**. **Click** here to verify…"
Correct: P(phishing) = 1.00

"…Kindly **reply** with your **bank** details so we can **transfer** your share of the **funds**"
Correct: P(phishing) = 0.99

Finding: the model learned that unknown words (<unk>) signal phishing, because spammers misspell words. Business mail full of names and codes then gets flagged. Fix: sub-word tokenisation (BPE).



Why deep learning didn't win here

1

The signal is in the words

Bag-of-words already captures “verify your account”; word order adds little

2

Small data for big models

12k training emails for 4M parameters with no pre-trained embeddings; the BiLSTM overfit

3

Truncation

Deep models read 300 tokens; the baseline made 0 errors on emails over 1,000 tokens

But they make different mistakes

37	42	58
errors, baseline	errors, TextCNN	errors, BiLSTM

Only 17 errors are shared by all three. Averaging their probabilities cancels much of the rest: **Ensemble F1 0.983, the best of all**

Takeaways and next steps

What we learned

- Deep models reach 98% accuracy, but a TF-IDF baseline is just as strong on this data
- An ensemble of all three is best: F1 0.983, 16 misses and 18 false alarms in 2,627 emails
- The threshold sets the FP/FN balance, and that is a policy decision
- Remaining errors come from the data: noisy labels, short texts, new attack styles

Next steps

- Fine-tune a pre-trained transformer (DistilBERT)
- Add sender, header and URL features (SPF/DKIM, link targets)
- Cleaner, more recent data that separates phishing from spam
- Calibrate probabilities so unsure cases go to a human

Live demo: `cd src && python predict.py --demo`

Thank you

Questions?

Code, trained models, figures and full report: README.md and report/REPORT.md in the project folder